

PCEP-30-0x Exam

Topics:

1. print
2. variables
3. datatypes
 - i. String
 - ii. Integer
 - iii. Float – decimal fractions
 - iv. Booleans (True or False)
4. Comments
5. Numerical Representations
 - i. Underscores in numbers – for easy readability
 - ii. Scientific Notations – e power -4
 - iii. Octal numbers – 0O or 0o
 - iv. Hexadecimal numbers – 0X or 0x
6. Operators
 - i. Addition
 - ii. Subtraction
 - iii. Multiplication
 - iv. Standard Division – returns float - /
 - v. Integer Division – returns integer - //
 - vi. Modulus Division – returns remainder - %
 - vii. Power operator – 2**3
7. Reassigning values
8. Input() function
9. Operator Precedence

Precedence Rank (High → Low)	Operator(s)	Type	Notes
1 (Highest)	<code>()</code>	Parentheses	Grouping — forces order.
2	<code>**</code>	Exponentiation	Right-to-left binding (important!).
3	<code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>	Multiplication, Division, Floor division, Modulus	Left-to-right binding.
4	<code>+</code> , <code>-</code>	Addition, Subtraction	Left-to-right binding.
5	<code>not</code>	Logical NOT	Higher precedence than <code>and</code> , <code>or</code> .
6	<code>and</code>	Logical AND	
7 (Lowest)	<code>or</code>	Logical OR	

10. Order of Operators

- i. `()`
- ii. `**` → The exponentiation operator (`**`) uses right-sided binding (i.e. Starts from the right)
- iii. `*` / `//` / `%`
- iv. `+` -

Precedence Rank	Operator	Description
1 (Highest)	<code>()</code>	Parentheses — used for grouping; they force certain operations to happen first.
2	<code>**</code>	Exponentiation — raises a number to the power of another (<code>2 ** 3 = 8</code>). It binds right-to-left (important!).
3	<code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>	Multiplication, Division, Floor Division, Modulus — they are left-to-right associative.
4 (Lowest among your list)	<code>+</code> , <code>-</code>	Addition and Subtraction — evaluated after multiplication/division types.

11. Operator Precedence

Operator	Precedence Rank	(Higher number means evaluated earlier)
<code>></code> , <code><</code> , <code>==</code> (comparison operators)	Higher (evaluated first)	
<code>and</code>	Middle	
<code>or</code>	Lower (evaluated later)	

12. Keyword Arguments or Named Arguments – always optional and always have a default value

- sep (default value is space)
- end (default value is new line)

13. Bit Operators: `~` `&` `^` `|` `<<` `>>`

14. Conditional Blocks and Loops

- If-elif-else blocks
- `=` is assigning values
- `==` checking equality
- Boolean Operators: `and`, `or`, `not`
- Priortiy:
 - `not`
 - `and`
 - `or`
- while loop
- for loop
- break and continue

15. Data Collections or Container Data Types

- Tuples
 - Dictionaries
 - Lists: Create List, add value to List, insert value to List at a specific position, delete value from List, delete List, iterating List
 - Strings
- List indices start at 0
 - Indexing: Example `list_variable[0]`
 - Slicing: example: `list_variable[0:2]`
 - Method are functions that belong to specific data
 - `list_name.sort()` → sorts the original list
 - `sorted(list_name)` → gives back a new sorted list and keeps the original list unchanged
 - Nested List is also called Multi-Dimensional List

- i. List methods: insert, append, sort
- j. Lists are mutable, Tuples are immutable, Strings are immutable in Python
- k. List is when you have many values of same datatype, same class
- l. Dictionaries are key value pairs
- m. Tuples can be indexed and sliced, just like lists

16. Functions

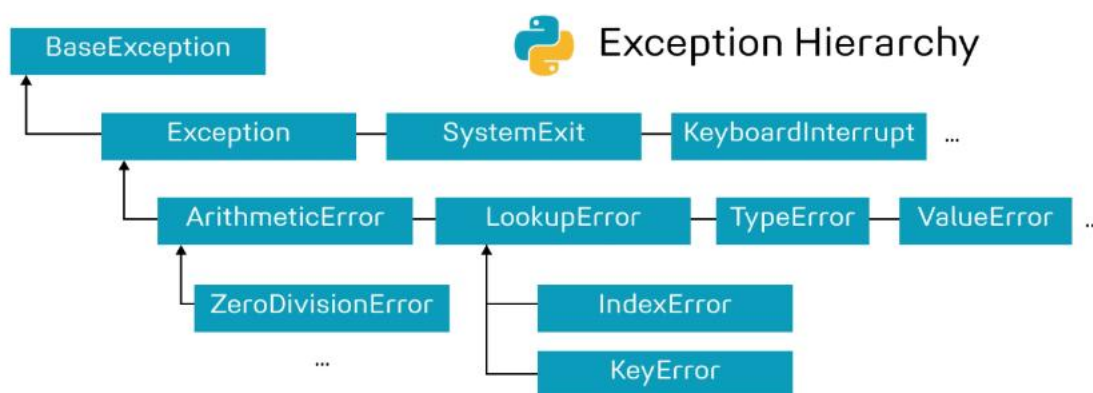
- a. function(positional_arguments or named_parameters)
- b. All positional arguments must appear before any named arguments
- c. Shadowing: Local variable inside the function shadows the global variable
- d. Functions can typically:
 - i. Cause some effect
 - ii. Return a meaningful value
- e. Recursion
- f. Generators: "yield" is the keyword

17. Difference between `x is None` and `x == None`. Functions without return or yield statements implicitly return None.

Aspect	<code>x is None</code>	<code>x == None</code>
Meaning	Check if <code>x</code> is exactly the same object as <code>None</code>	Check if <code>x</code> is equal to <code>None</code> (value-wise)
Type of comparison	Identity comparison (are they literally the same object)	Value comparison (do they look equal)
Speed	Slightly faster	Slightly slower
Recommended?	✅ Yes (best practice)	❌ Not recommended (but works)

18. Exceptions

- a. An exception is an event which occurs during the execution of a program that disrupts the normal flow of the program instructions.
- b. `SyntaxError`
- c. Exception Hierarchy
- d. Propagating Exceptions
- e. Assertion Exceptions: Should not be used in real-life, production-ready code



(*) only shows selected exception types

```
Command Prompt - python  X + v
Microsoft Windows [Version 10.0.26100.3775]
(c) Microsoft Corporation. All rights reserved.

C:\Users\YACE>python
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun  6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
>>>
```